

Projet : Activation de capteurs pour la surveillance de zones

Groupe : MEYER Timothée / FROEHLY Jean-Baptiste

Sujet et problématique initiale

Nous partons donc d'un réseau de **capteurs**, ici vidéos, déployés pour surveiller plusieurs **zones cibles**. Nous avons donc trois contraintes de départ :

1. **Couverture partielle** : Chaque capteur ne peut surveiller qu'un certain nombre de zones spécifiques.
2. **Énergie limitée** : Chaque capteur fonctionne sur une batterie qui a une durée de vie maximale.
3. **Surveillance continue** : Toutes les zones cibles doivent être surveillées en permanence (il ne doit y avoir aucune zone "aveugle").

Notre objectif est donc d'organiser un planning d'activation de ces capteurs (quels capteurs allumer ensemble et pendant combien de temps) afin que la surveillance globale dure **le plus longtemps possible** avant l'épuisement des batteries.

Autrement dit, notre objectif est de maximiser la durée de vie du réseau de capteurs.

Concepts Clés

1. La "configuration"

- **Une configuration valide** est un groupe de capteurs allumés en même temps qui permet de surveiller toutes les zones cibles.
- **Elle est "élémentaire" (ou minimale)** s'il n'y a aucun capteur inutile à l'intérieur. C'est-à-dire que si on éteint ne serait-ce qu'un seul capteur du groupe, au moins une zone n'est plus surveillée. On cherche absolument des configurations élémentaires pour éviter de vider la batterie de capteurs superflus.

2. Le problème du nombre de combinaisons

Il est facile avec seulement 4 capteurs, de trouver les bons groupes à la main. Mais avec 100 ou 1000 capteurs, le nombre de combinaisons possibles est astronomique. Un ordinateur ne peut donc pas toutes les tester une par une. Il faut donc utiliser deux étapes intelligentes pour résoudre le problème.

Étape 1 : Trouver les bonnes combinaisons (La Recherche Tabou)

Pour trouver un grand nombre de configurations valides et minimales, nous utilisons l'algorithme de **Recherche Tabou**. C'est une méthode d'exploration étape par étape :

1. **Le point de départ** : On commence par allumer un groupe de capteurs au hasard.
2. **Le déplacement** : On modifie ce groupe en allumant ou en éteignant un seul capteur. On choisit le changement qui améliore le plus la couverture tout en utilisant le moins de capteurs possible.
3. **La règle "Tabou"** : Pour éviter de tourner en rond (par exemple, éteindre un capteur, le rallumer à l'étape suivante, puis le réteindre...), on interdit temporairement de modifier à nouveau les capteurs que l'on vient de changer.
4. **La simplification** : Dès que le groupe actuel couvre toutes les zones, on cherche à l'épurer : on éteint un par un les capteurs qui ne sont pas indispensables pour obtenir une configuration élémentaire.
5. **La diversification** : On répète ce processus 100 ou 200 fois en partant de configurations initiales différentes et en parallèle pour accélérer les calculs.

Étape 2 : Planifier les durées d'activation (La Programmation Linéaire)

Une fois que nous avons notre liste de configurations élémentaires valides, nous devons décider **combien de temps faire fonctionner chaque configuration**.

Nous formulons cela sous forme de **programme linéaire** :

- **Ce que l'on cherche** : Le temps d'activation t de chaque configuration (par exemple: la configuration A fonctionne pendant 2 heures, la configuration B pendant 3,5 heures).
- **L'objectif** : Maximiser la somme de ces temps (la durée de vie totale de la surveillance du réseau).
- **Les contraintes** : Pour chaque capteur individuel, la somme des temps d'activation de toutes les configurations auxquelles il participe ne doit pas dépasser sa durée de vie maximale de batterie.

Le solveur GLPK calcule ce planning optimal en une fraction de seconde.

Les résultats des tests

Nous avons exécuté notre programme sur les 5 instances de test fournies :

Fichier Instance	Capteurs (N)	Zones (M)	Configs Générées	Configs Activées	Durée de vie optimale (T^*)	Temps de calcul
0_petit_test.txt	4	3	4	4 (100 %)	8.50 unités	0.81 s
1_moyen_test_1.txt	20	10	130	13 (10 %)	104.00 unités	0.82 s
1_moyen_test_2.txt	10	10	18	9 (50 %)	395.00 unités	0.90 s
2_gros_test.txt	100	200	515	50 (9.7 %)	1922.17 unités	2.01 s
3_maxi_test.txt	1000	500	1947	~92 (4.7 %)	3333.28 unités	49.83 s

Détail de l'ordonnancement pour 0_petit_test.txt

Pour cette petite instance à 4 capteurs avec des batteries de (6h, 3h, 2h et 6h), le solveur a planifié :

- Activer la configuration (Capteurs 2 et 4) pendant **2.5 unités de temps**.
- Activer la configuration (Capteurs 1 et 2) pendant **0.5 unité de temps**.
- Activer la configuration (Capteurs 1 et 3) pendant **2.0 unités de temps**.
- Activer la configuration (Capteurs 1 et 4) pendant **3.5 unités de temps**.

Durée de vie optimale obtenue : 8.5 unités de temps.

On constate ici une **efficacité énergétique parfaite de 100 %** : l'intégralité des batteries de tous les capteurs du réseau a été consommée sans aucun gaspillage (6.0 pour s_1 , 3.0 pour s_2 , 2.0 pour s_3 et 6.0 pour s_4).

Ce qu'on en retient

1. Le rôle de filtre du Programme Linéaire

Dans les grandes instances, seule une infime minorité des configurations élémentaires générées est réellement activée par le solveur (par exemple, seulement **10 %** pour `moyen_test_1` et **4.7 %** pour `maxi_test`). Le programme linéaire joue un rôle de filtre : il élimine les configurations moins efficaces et ne sélectionne que la combinaison de sous-ensembles la plus complémentaire pour maximiser la durée de vie globale du réseau.

2. Influence du nombre de configurations élémentaires

La taille du pool de configurations dépend directement du nombre d'itérations et de redémarrages de la recherche tabou. Nous avons fais des tests sur l'instance des tests "moyen 1" en faisant varier le nombre de redémarrages (restarts) de l'heuristique :

Nombre de Redémarrages	Taille du Pool de Configs	Durée de vie optimale (T^*)
1	9	54.00
2	27	100.00
5	26	96.00
10	42	104.00
50	100	104.00
100	127	104.00

- **Observation de convergence** : Avec un seul démarrage, le pool est trop restreint et le solveur est bridé à une durée de vie de **54.00**. Augmenter la recherche permet d'enrichir le pool et de trouver de bien meilleures combinaisons. La convergence vers l'optimum absolu de **104.00** est atteinte dès 10 redémarrages (42 configurations). Pousser la recherche à 100 redémarrages génère plus de configurations (127) mais n'améliore plus la durée de vie, montrant qu'un nombre modéré de restarts suffit à garantir l'optimalité globale tout en économisant le temps de calcul.

3. Densité du réseau et facteur multiplicateur de durée de vie

Si l'on compare `moyen_test_1` (20 capteurs, 10 zones, batterie moyenne $\approx$ 21 unités) et `moyen_test_2` (10 capteurs, 10 zones, batterie moyenne $\approx$ 87 unités) :

- Pour `moyen_test_1`, le solveur parvient à atteindre **104.00 unités** (soit environ **5 fois** la batterie moyenne d'un capteur).
- Pour `moyen_test_2`, la durée de vie s'élève à **395.00 unités** (soit environ **4.5 fois** la batterie moyenne). Cela montre que l'ordonnancement intelligent par programmation linéaire permet de multiplier de manière similaire la durée de vie moyenne des capteurs d'un facteur de **4.5x à 5x**, démontrant l'efficacité du découpage temporel des activations par rapport à une activation simultanée brute de tous les capteurs.

4. Importance fondamentale de l'Éléментарité

Les configurations non élémentaires (qui contiennent des capteurs superflus) sont nuisibles. Si une configuration n'est pas minimale (par exemple si elle active 3 capteurs alors que 2 suffisent), le capteur supplémentaire vide sa batterie inutilement. L'étape de minimalisation gourmande intégrée dans notre recherche tabou est donc critique : elle empêche le gaspillage d'énergie et restreint le problème linéaire aux seules variables énergétiquement viables.